

Carolus / RoboMaster S1 — Complete Technical Manual

August 10, 2026

Abstract

This manual is self-sufficient: following it from Chapter 3 onward requires no other document, inherited tutorial, or vendor PDF. It covers the complete path from unboxing a rooted RoboMaster S1 and a Raspberry Pi 4B to a running Carolus/uVGS-2 pose-estimation pipeline, then to driving the robot and running its beacon-docking behaviour (Chapter 10), using only the corrected, currently-working configuration of this project — not the historical or inherited versions of any script, launch file, or procedure. Every program listed here is the version that actually runs today.

Already set up? Chapter 2 is the five-minute routine for a normal session.

Contents

1	Introduction	2
1.1	What this manual covers	2
1.2	Hardware	2
1.3	Software stack	2
1.4	How the pieces fit together	3
1.5	What Carolus is	3
1.6	The four operating states	4
1.7	How to use this manual	4
2	Quick Start — A Typical Session	5
2.1	The five-minute version	5
2.2	Without the launcher, by hand	5
2.3	Is it working? Three checks from the lab PC	6
2.4	Shutting down	6
3	Powering On the Robot	7
3.1	Before you start	7
3.2	Power-on sequence	7
3.3	Confirming the robot is powered on	8
3.4	Confirming the robot is rooted	8
3.5	Which camera is mounted	8
4	Rooting the S1	9
4.1	What the hack does	9
4.2	Step-by-step procedure	9
4.2.1	What you need	9
4.2.2	Procedure	10
4.3	A pitfall to know before rooting	11
5	Setting Up the Raspberry Pi	12
5.1	Powering the Pi — two USB-C ports, only one of them right	12
5.2	Flashing Ubuntu Server	12
5.3	Desktop environment (optional)	13
5.4	Enabling headless SSH access	13
5.5	Building Python 3.7 from source	14

5.6	Virtual environment and the RoboMaster SDK	14
5.7	Installing ROS Noetic	15
5.8	Getting this project’s code onto the Pi	15
5.9	A pitfall to expect at first launch	16
6	Network Setup (RNDIS)	17
6.1	Bringing up the link	17
6.1.1	If you intend to automate this over SSH	17
6.2	Verifying the link	18
6.3	A quick automated alternative	18
6.4	Lab PC hostname resolution	18
6.5	Summary of addresses used in this manual	18
7	The Camera and Robot Bridge	19
7.1	Design: a single SDK connection	19
7.2	Starting the bridge	19
7.3	What it connects to	19
7.4	Verifying it is running	21
7.5	Gimbal yaw: two signals, two different meanings	21
7.6	The internal camera vs. an external D455	21
8	Camera Calibration	22
8.1	Current values in use	22
8.2	Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox	22
8.3	Alternative / complementary method: Kalibr	23
8.3.1	Target	23
8.3.2	Installing Kalibr	23
8.3.3	Capturing data and running the calibration	24
8.3.4	Reading the result	24
8.4	Camera–IMU time offset	24
9	Building and Launching Carolus	25
9.1	Where Carolus runs	25
9.1.1	The two machines, side by side	26
9.1.2	Measured, 2026-08-04 — and what the numbers do <i>not</i> say	26
9.1.3	A note on visual-inertial navigation	27
9.2	Getting this project’s code onto the machine that will build it	27
9.2.1	Setting up an SSH key	28
9.3	Building on Ubuntu 20.04 (the Raspberry Pi — normal case)	28
9.4	Building on Ubuntu 22.04 (lab-PC fallback only)	29
9.4.1	Fix 1: glibc symlinks	29
9.4.2	Fix 2: libpython3.8 symlink	29
9.4.3	Fix 3: Boost 1.71 (real libraries, not symlinks)	29
9.4.4	Fix 4: ICU 67	30

9.4.5	Building, after the four fixes	30
9.4.6	Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)	30
9.4.7	If an additional ROS package needs to be built from source	30
9.5	Launching Carolus	31
9.5.1	The hardware profile	31
9.5.2	Launching	32
9.5.3	The pose timestamp: <code>stamp_from_acquisition</code>	32
9.6	Verifying it is running	33
9.7	Beacon geometry and blob-filtering parameters	33
10	Operating the Robot: the Launcher	35
10.1	The launch tabs, in order	35
10.2	Driving the robot by hand	36
10.3	The main controls	37
10.4	Session logs on disk	37
10.5	Shutting down, and why it matters	37
10.6	Beacon docking (T5)	38
11	Troubleshooting	39
11.1	Nothing responds: the robot ignores every command	39
11.2	The Pi will not turn on	39
11.3	<code>rostopic</code> from the lab PC finds nothing	39
11.4	The camera publishes but <code>/pose</code> never does	40
11.5	The keyboard does not drive the robot	40
11.6	<code>/imu</code> is silent	40
11.7	The build fails	40
11.8	Messages you can safely ignore	40

Chapter 1

Introduction

1.1 What this manual covers

This is the complete, self-contained setup and operation manual for the Carolus/RoboMaster S1 platform: a DJI RoboMaster S1 ground robot, rooted to expose the RoboMaster EP SDK, carrying a Raspberry Pi 4B at the turret position, running Carolus (also called uVGS-2) — a 4-LED beacon pose-estimation system — to autonomously search for, align to, approach, and dock at a fixed beacon.

Everything in this manual has been executed and verified on this exact hardware. Where a step has not been physically re-verified on this specific unit but is inherited from a documented, working precedent (e.g. a rooting procedure that predates this project), that is stated explicitly at the point it occurs.

1.2 Hardware

- **Robot:** DJI RoboMaster S1, rooted via the `s1_sdk_hack_v0.0.5` community hack, which exposes the RoboMaster EP SDK (chassis drive, gimbal control, onboard camera, wheel odometry, IMU) over a local API not officially released for the S1.
- **Companion computer:** Raspberry Pi 4B, mounted at the turret, connected to the robot over USB via RNDIS. The Pi holds the single connection to the robot’s SDK, runs the camera/beacon-tracking node, and **runs Carolus itself**, so the full perception pipeline is onboard and the robot needs no network link to estimate its pose (Section 9.1).
- **Lab PC:** runs the operator GUI and builds/edits the same Carolus code for development, but does not operate the robot with it — not physically connected to the robot, communicating with the Pi over the local network.
- **Camera:** the S1’s own internal camera (built into the gimbal/head), streamed over the SDK. This project does not use an external Intel RealSense D455 — some inherited reference material describes a D455-based pipeline (see Chapter 8’s calibration discussion); that material is noted where it applies to calibration *technique*, not to this robot’s actual camera hardware.

1.3 Software stack

- **Operating systems:** Raspberry Pi runs Ubuntu 20.04.6 LTS (aarch64). The lab PC runs Ubuntu 22.04.5 LTS (x86_64) — not the officially supported target for ROS Noetic, which requires local

workarounds documented in Chapter 9.

- **Middleware:** ROS Noetic (1.17.4) on both machines.
- **Robot control:** the official `robomaster` Python SDK, requiring Python 3.7 specifically.
- **Vision:** `Carolus/uVGS-2`, a C++ ROS node built on OpenCV, Eigen3, and Ceres Solver, performing 4-LED blob detection and a Perspective-4-Point (P4P) pose solve.

1.4 How the pieces fit together

Three machines, one data path. The robot never talks to the lab PC directly — everything goes through the Pi, which holds the only SDK connection.

RoboMaster S1		Raspberry Pi 4B		Lab PC
camera, chassis, gimbal	USB / RNDIS	SDK bridge, Carolus, ROS master	local net	launcher (GUI), development
[]	<----->	[]	<----->	[]
192.168.42.2		192.168.42.3 / 192.168.0.103		192.168.0.100

Inside the Pi, the pipeline is a straight line from pixels to a pose:

camera frames	-->	blob detection	-->	P4P solve (Ceres)	-->	/pose 6-DOF	-->	control SEARCH / ALIGN / APPROACH
		4 LEDs						
		+						+
				/carolus/cmd_vel		->	back to the robot	

`rm_cam_beacon.py` (Chapter 7) owns both ends — it publishes the frames and it is the only thing that sends motion commands to the robot. `Carolus` (Chapter 9) owns the middle.

1.5 What Carolus is

`Carolus` (internally named `uVGS-2`, Unified Visual Guidance System version 2) is a monocular pose-estimation system originally developed for NASA’s *Astrobee* free-flying robot aboard the International Space Station, derived from the earlier `SVGS` system. It detects four illuminated beacon points (LEDs) in a camera image, identifies which detected blob corresponds to which known 3D point on the physical beacon, and solves a Perspective-4-Point optimization (via Ceres Solver, not OpenCV’s `solvePnP` family) to recover the beacon’s full 6-DOF pose relative to the camera.

Carolus’s own native coordinate frame is left-handed. This is not obvious from the code and is stated here because it matters for every transform downstream: converting a `Carolus` pose or orientation into ROS’s right-handed convention (REP-103) requires a true change-of-basis on the rotation, not a component-wise permutation of a quaternion’s vector part — the latter is only valid for a proper (determinant +1) rotation, and a left-to-right-handed conversion is improper (determinant −1). The correct method (quaternion composition, not permutation) is given at the point it is needed in this manual.

1.6 The four operating states

Once running, the robot’s autonomous behavior is a state machine:

SEARCH The gimbal sweeps without moving the chassis, looking for the beacon. If not found from the current heading, the chassis rotates to a new heading and the gimbal sweeps again (a “fan search” pattern).

ALIGN Once the beacon is detected, the chassis rotates in place until it faces the beacon.

APPROACH The gimbal tracks the beacon while the chassis advances along the same heading.

STOP Triggered once within a target stopping distance of the beacon.

A separate, more deliberate **docking** behavior performs a calibrated, verified approach-and-stop sequence rather than the lighter continuous SEARCH/ALIGN/APPROACH loop (Section 10.6).

1.7 How to use this manual

If the hardware is already set up, go straight to Chapter 2 — it is the five-minute routine for a normal session, and it is what you will use every day after the first one.

From scratch, follow the chapters in order: power the robot on and confirm it is rooted (Chapter 3), root it if it is not (Chapter 4), set up the Raspberry Pi (Chapter 5), bring up the network link (Chapter 6), bring up the camera pipeline (Chapter 7), calibrate the camera (Chapter 8), build and launch Carolus (Chapter 9), then drive the robot (Chapter 10).

Do not calibrate the camera on a first run. The values already in the repository work on this robot. Chapter 8 is for a different camera, or for when poses look physically wrong — skipping it initially is the right call, not a shortcut.

What you need, and how long it takes

Stage	Time	What it needs
Rooting the S1 (Ch. 4)	~30 min, once ever	a Windows or macOS machine, micro-USB cable, Android Platform-Tools, the RoboMaster app
Raspberry Pi from blank (Ch. 5)	half a day — building Python 3.7 from source is the long part	Pi 4B, SD card (≥ 16 GB), card reader, keyboard + monitor for first boot, Ethernet cable
Network, camera, Carolus build (Ch. 6–9)	1–2 h, mostly compiling	USB-C cable (Pi↔robot), the lab PC on the same network
A normal session afterwards (Ch. 2)	5 min	a charged robot battery, a powered beacon

Everything above except the rooting step runs from Linux. The rooting step is the one exception, and it is needed only once per robot.

Every command, parameter value, and code listing in this manual is the *current, corrected* version — the one that actually runs on this project today.

Chapter 2

Quick Start — A Typical Session

Runs on: the **lab PC** (the launcher and these commands), driving the **Raspberry Pi** over SSH. The robot must be powered on.

This chapter is for a machine that is already set up. If you are starting from bare hardware, skip it and follow Chapters 3–9 in order, then come back here — this is what every session after the first one looks like.

2.1 The five-minute version

1. **Power on the robot** — listen for **two chimes**, not one (Chapter 3). One chime means the root hack is not active and nothing below will work.
2. **Power on the Pi** with the button on the UPS HAT (Section 5.1 — the HAT’s port, not the Pi’s).
3. **Bring up the RNDIS link**, on the Pi. It does not survive a reboot, so this is needed every session (Chapter 6):

```
sudo ip link set eth1 up
sudo ip addr add 192.168.42.3/24 dev eth1
ping -c 4 192.168.42.2          # the robot -- expect 0% loss
```

4. **Start the launcher**, on the lab PC:

```
python3 shortcuts/carolus_launcher.py
```

5. **Click the tabs in order** (Chapter 10): T1, then T2, then T3. Each one waits for the previous.

That is a working stack: camera streaming, Carolus solving poses, the robot drivable from the keyboard.

2.2 Without the launcher, by hand

Useful when debugging, or when the launcher itself is the suspect. Three separate SSH shells to the Pi, in this order — each needs its own `ROS_MASTER_URI/ROS_IP`, they are per-shell:

```
# --- Shell 1, on the Pi: the ROS master ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
roscore

# --- Shell 2, on the Pi: the camera + SDK bridge (Chapter 6) ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/Python-3.7.17/env/bin/activate
python3.7 carolus_ws/src/robomaster_cam/scripts/rm_cam_beacon.py

# --- Shell 3, on the Pi: Carolus itself (Chapter 8) ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/carolus_ws/devel/setup.bash
roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false
```

2.3 Is it working? Three checks from the lab PC

Run these on the **lab PC**, not the Pi — that is the whole point: they confirm the nodes are visible across the network, not just locally.

```
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.100

rostopic hz /camera/color/image_raw    # expect ~16-30 Hz
rostopic hz /pose                      # ~13 Hz, ONLY with a beacon in view
roslaunch carolus_node testcarolus.launch # expect /ep_camera_beacon_follower
```

If `rostopic` finds nothing while the Pi says the node is running, the cause is almost always a missing `ROS_MASTER_URI`/`ROS_IP` in the shell that launched it — `rospy` then falls back to `localhost` and the node is invisible from anywhere else.

2.4 Shutting down

Close the launcher — it kills its own remote processes and verifies they died. By hand, on the Pi, the bracket in each pattern is not optional (see Chapter 10):

```
killall -f "[r]m_cam_beacon"
killall -f "[r]oscore"; killall -f "[r]osmaster"
pgrep -af "roscore|rosmaster|rm_cam_beacon"    # confirm nothing is left
```

Always confirm the bridge is dead before the next session. An orphaned `rm_cam_beacon.py` keeps the robot's single SDK connection open, and the next launch then hits the double-connection failure — motion commands stop working with no error shown (Chapter 7).

Chapter 3

Powering On the Robot

Runs on: the **robot itself** (physical handling). The Raspberry Pi and lab PC are not involved until Chapter 5.

3.1 Before you start

- The S1’s smart battery, charged. Charging is required before first use even if the battery shows some charge — this “wakes up” the battery.
- The S1 charger, plugged into mains (100–240 V, 50/60 Hz). A full charge takes approximately 1 h30.
- The battery only charges between 5 °C and 45 °C.
- Do not touch the motors, motor mounting plate, or the inside of the Mecanum wheels immediately after powering the robot off — they can be hot.
- Do not attempt to charge a battery that has dropped below 1 V per cell; this poses a fire risk.
- Use the official DJI charger only.

3.2 Power-on sequence

The battery compartment is behind the chassis’s rear armor panel, so the sequence is: open, insert, power on, close.

1. Press the rear armor release button to open the chassis’s rear cover.
2. Insert the smart battery into its compartment, making sure it is firmly seated — a poorly seated battery gives poor contact and can cause the robot to lose battery information.
3. Press and hold the power button (on the battery itself) for more than two seconds to power on.
4. Close the chassis’s rear armor.

The Raspberry Pi at the turret is a separate power domain from the robot chassis; there is no required power-on order between the two, and the USB/RNDIS cable connecting them is for the SDK link (Chapter 6), not for powering either device.

3.3 Confirming the robot is powered on

Expected light signals at power-on:

- **Gimbal (head) LED:** blinks cyan, counterclockwise.
- **Chassis LED:** solid cyan.
- **Motion controller LED:** blinks blue slowly — this means the motion controller is working normally.

3.4 Confirming the robot is rooted

This project assumes a rooted S1 exposing the RoboMaster EP SDK. Without the root hack (Chapter 4), SDK commands (chassis, gimbal, camera) will not respond, even though the robot otherwise powers on and behaves normally through the stock RoboMaster app.

Audible cue at boot: a correctly rooted S1 emits two chimes instead of the usual single chime. The hack persists across power cycles — it does not need to be reapplied every time the robot is powered on.

Concrete test, once the Pi is connected over RNDIS (Chapter 6): the camera/beacon node (`rm_cam_beacon.py`, Chapter 7) opens the one and only SDK connection this project uses:

```
1 self.ep = robot.Robot()  
2 self.ep.initialize(conn_type="rndis")
```

- **Success:** the node's log shows the connection succeeding, and the robot responds to `Twist` commands published on `/carolus/cmd_vel` (Chapter 7).
- **Failure:** the SDK connection fails or times out. First confirm the network link itself (ping 192.168.42.2 from the Pi, Chapter 6). If the network link is fine but the SDK connection still fails, redo the power-on and listen carefully for the double chime — a single chime means the root hack is not active.

Do not use any script other than `rm_cam_beacon.py` to test this. Two independent scripts each attempting `ep.initialize(conn_type="rndis")` at the same time silently blocks robot motion after a few commands, with no error surfaced — this failure mode is why this project uses exactly one script holding the SDK connection at any time (see Chapter 7).

3.5 Which camera is mounted

This project uses the S1's own internal camera, not an external Intel RealSense D455. Physically: a D455 is an external rectangular camera housing mounted in addition to the robot's own head; if there is no such external camera, the internal camera is in use. In software, once `rm_cam_beacon.py` is running (Chapter 7), the internal camera feed appears on `/camera/color/image_raw` immediately, with no separate D455 driver (`rs_camera.launch`) needed.

Chapter 4

Rooting the S1

Runs on: a **Windows or macOS computer** connected to the robot by micro-USB. Not the Raspberry Pi, not the Ubuntu lab PC — the RoboMaster app required for this step does not run on Linux.

If Chapter 3’s double-chime check already confirmed the robot is rooted, this chapter is informational only — do not re-run it.

4.1 What the hack does

DJI’s RoboMaster app exposes a limited, sandboxed Python scripting environment (“the Lab”). The S1’s Python subsystem runs with root privileges internally, but DJI restricts which modules a Lab script can import via a whitelist attached to the `__builtins__` object. Several of the whitelisted modules (needed for legitimate scripting features) still hold a reference to the real, unrestricted `__builtins__` internally. That reference can be recovered and used to import `subprocess`, which can then invoke a shell script already present on the robot’s filesystem (`adb_en.sh`) to enable `adb` and open a root shell over USB.

Once a root shell is available, a patch is pushed to the robot’s filesystem that installs the RoboMaster EP SDK’s full command set (the EP is DJI’s higher-end sibling to the S1; underneath, the two share nearly identical firmware) onto the S1, which DJI has never officially released to S1 owners.

Warning, carried over from the hack’s own documentation, to be taken literally: this has the potential to break the robot. It did not break this project’s unit, but there is no guarantee for any given unit.

4.2 Step-by-step procedure

4.2.1 What you need

- A computer running Windows or macOS (the RoboMaster app requires one of these to configure the S1; the Raspberry Pi is not involved in this chapter).
- Android SDK Platform-Tools (`adb.exe` and friends) — unzip anywhere, remember the path.
- A micro-USB cable.
- The RoboMaster app, with a DJI account.

4.2.2 Procedure

1. Connect the computer to the S1's Intelligent Controller Micro-USB port with the micro-USB cable. Ensure no other Android device is connected over USB during this procedure.
2. Power on the robot (Chapter 3) and connect the RoboMaster app to it via its own Wi-Fi hotspot (visible on top of the gimbal, with a password shown alongside it).
3. In the app: **Lab** → **DIY Programming** → select **Python** on the top bar → **New** to create a new program.
4. Paste the following into the program editor exactly. This is the current, working root script for this hack version — do not use an older variant that imports `'subprocess'` as a literal string or reads from `rm_log` instead of `rm_define`; a firmware update closed that specific path.

```
1 def root_me(module):
2     __import__=rm_define.__dict__['__builtins__']['__import__']
3     return __import__(module, globals(), locals(), [], 0)
4
5 builtins=root_me('builtins')
6 x=root_me('sub'+ 'process')
7 proc=x.Popen('/system/bin/adb_en.sh', shell=True, executable='/system/bin/
8     sh',
9             stdout=x.PIPE, stderr=x.PIPE)
```

5. Run the program from within the Lab. A correct run shows no compilation errors and the console prints **Execution Complete**. Do not close the RoboMaster app.
6. On the computer, open a terminal in the folder containing the Android Platform-Tools and run:

```
adb devices
```

The S1 should appear in the device list.

7. Confirm root access:

```
adb shell
```

8. With the root shell open, from a second terminal, in the folder containing the hack's distribution files (`dji_scratch`, `dji.json`, `dji_hdvt_uav`, `dji_sys`, `patch.sh`), push the patch:

```
adb shell rm -rf /data/dji_scratch
adb push dji_scratch /data/.
adb push dji.json /data/.
adb push dji_hdvt_uav /data/.
adb push dji_sys /data/.
adb shell chmod 755 /data/dji_hdvt_uav
adb shell chmod 755 /data/dji_sys
adb push patch.sh /data/.
```

9. Restart the S1 fully (hold the power button until it turns off, then hold it again to turn back on). On boot, listen for **two chimes instead of one** — this confirms the hack is active (Chapter 3's check).
10. Close the RoboMaster app and power off the robot; the setup for this chapter is complete.

4.3 A pitfall to know before rooting

Starting with hack version 0.0.3, `adb` is re-enabled by default in the patch, which **disables RNDIS support** — the exact link this project’s Pi↔robot connection depends on (Chapter 6). If RNDIS stops working after rooting, comment out the `adb` line in `patch.sh` before pushing the patch, or re-push a corrected `patch.sh` and restart the robot again.

Chapter 5

Setting Up the Raspberry Pi

Runs on: the **Raspberry Pi 4B** (mounted on the robot), with a keyboard/monitor for first boot or over SSH from the lab PC afterwards.

This chapter takes a blank Raspberry Pi 4B to a working Ubuntu 20.04 installation with Python 3.7, the RoboMaster SDK, ROS Noetic, and SSH access. If the Pi is already set up, skip to Chapter 6.

5.1 Powering the Pi — two USB-C ports, only one of them right

Read this before plugging anything in. The Pi carries a UPS HAT (a green PCB underneath it, with its own battery and a physical ON/OFF button on the back). That means **two different USB-C ports**, which look interchangeable and are not:

- **The HAT’s USB-C port** (on the green PCB underneath) — **this is the one to use.** It charges the internal battery, a blue “Charge” LED lights up, the ON/OFF button works, and the Pi then runs standalone on battery.
- **The Pi’s own POWER IN port** (on the Pi board itself, to the left of the HDMI connectors) — powers the Pi directly. It boots, but the HAT’s ON/OFF button is ignored and the Pi is tethered to the cable. Not for normal sessions.

If the Pi will not turn on: check the cable is on the HAT port and not the Pi’s POWER IN, and that the charge LED is blue. This is the single most common physical blocker on this setup.

Normal routine: plug the charger into the HAT port the evening before a session. The next day the Pi runs on its own battery, started and stopped with the physical button — no cable, no tether.

5.2 Flashing Ubuntu Server

1. Download the Ubuntu Server ARM64 image from <https://cdimage.ubuntu.com/releases/focal/release/ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz>.
2. Decompress it:

```
unxz ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz
```


3. Identify the SD card device (do this carefully — writing to the wrong device destroys data on it):

```
lsblk
```

4. Flash it:

```
sudo dd if=ubuntu-20.04.5-preinstalled-server-arm64+raspi.img of=/dev/sdX  
bs=4M status=progress conv=fsync
```

5. Boot the Pi with the card inserted, plus keyboard, mouse, display, and an Ethernet cable for first boot. Default credentials: `ubuntu/ubuntu` (you will be asked to change the password on first login).

5.3 Desktop environment (optional)

Only needed if you want a graphical session on the Pi itself (not required for this project's normal operation, which runs headless):

```
sudo apt update  
sudo apt install ubuntu-desktop  
sudo reboot
```

5.4 Enabling headless SSH access

If SSH was not already enabled at flash time (Raspberry Pi Imager's advanced options support this directly), enable it on first boot:

```
sudo systemctl enable ssh  
sudo systemctl start ssh  
sudo systemctl status ssh    # optional: confirm it's running
```

Find the Pi's IP address on the local network:

```
hostname -I
```

Connect from the client machine, first with a password, then switch to key-based login:

```
# First connection (password):  
ssh <username>@<pi_ip>  
  
# On the client: generate a key  
ssh-keygen -t ed25519 -C "client-key"  
  
# Copy it to the Pi  
ssh-copy-id <username>@<pi_ip>  
  
# Test key-based login (no password prompt expected)  
ssh <username>@<pi_ip>
```

A convenient alias, in `~/.ssh/config` on the client:

```
1 Host raspberry  
2   HostName <pi_ip>
```

```
3 User <username>
4 IdentityFile ~/.ssh/id_ed25519
```

Note the exact directive name: `IdentityFile`, not `IdentifyFile` — a common typo that OpenSSH silently rejects the config over.

Connect with `ssh raspberry` from then on.

Optional hardening, once key-based login is confirmed working:

```
# Firewall
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw enable

# Disable password authentication (only after keys work!)
sudo nano /etc/ssh/sshd_config
# Ensure: PasswordAuthentication no
#         PubkeyAuthentication yes
sudo systemctl restart ssh
```

5.5 Building Python 3.7 from source

The RoboMaster SDK requires Python 3.7 specifically; Ubuntu 20.04 ships a newer default Python, so 3.7 is built from source into a separate location rather than replacing the system Python.

```
sudo apt update
sudo apt install -y build-essential libssl-dev zlib1g-dev \
    libreadline-dev libsqlite3-dev libbz2-dev libexpat1-dev \
    liblzma-dev tk-dev libffi-dev uuid-dev

cd ~/Python-3.7.17/
./configure --enable-optimizations
make -j4
sudo make altinstall
```

Using `altinstall` (not `install`) is deliberate: it installs as `python3.7` alongside the system Python, without overwriting it.

Confirm the SSL module built correctly (a common source of a broken venv later if skipped):

```
python3.7 -c "import ssl; print(ssl.OPENSSL_VERSION)"
```

5.6 Virtual environment and the RoboMaster SDK

Still in `~/Python-3.7.17/` from the previous section — the venv is created there, not at the home directory root, and every later reference to it in this manual (Chapter 7) uses the full path for exactly that reason:

```
python3.7 -m venv env
source env/bin/activate
pip install --upgrade pip
pip install robomaster numpy
pip install "opencv-python==4.5.5.64"
```

The pinned OpenCV version is a historically validated compatible version for this SDK/Python combination on this hardware; it is not a hard requirement enforced elsewhere in this manual, and later versions have since been used successfully on the lab PC (Chapter 9) without issue.

5.7 Installing ROS Noetic

```
sudo sh -c 'echo deb http://packages.ros.org/ubuntu\
$(lsb_release -sc)_main"> /etc/apt/sources.list.d/ros-latest.list'
sudo apt install curl
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \
| sudo apt-key add -
sudo apt update

# Full desktop version:
sudo apt install ros-noetic-desktop-full
# --- or, if the Pi struggles with the desktop version's resource use ---
sudo apt install ros-noetic-ros-base

echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc

sudo apt install python3-rosdep python3-rosinstall \
python3-rosinstall-generator python3-wstool build-essential
sudo rosdep init
rosdep update
```

On this project's Pi, the lighter `ros-noetic-ros-base` is what actually runs day to day — the Pi only needs to run `rm_cam_beacon.py` (Chapter 7), not a full desktop ROS toolchain.

5.8 Getting this project's code onto the Pi

The Pi needs this project’s own workspace — specifically `carolus_ws/src/robomaster_cam/`, which holds `rm_cam_beacon.py` (Chapter 7) and its companion scripts. Clone the project’s repository directly onto the Pi:

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git carolus_ws
```

This uses an SSH key registered against *your own* GitHub account (Chapter 9 covers generating and registering one). The repository is public, so no collaborator access is needed to read it — this only matters the moment you are given write access. Either way, **do not paste a shared personal access token into the clone URL**: it grants whoever reads it the full rights of the account that made it, and cannot be traced back to an individual. If SSH is not set up, an HTTPS clone works with no credentials at all:

```
git clone https://github.com/Arracheur2Bonnet/robotmaster.git carolus_ws
```

No build step is needed on the Pi side for `robomaster_cam` — it is a pure-Python package, run directly (Chapter 7).

5.9 A pitfall to expect at first launch

The RNDIS network interface (Chapter 6) does not come up automatically at Pi boot, even once everything above is configured correctly. This is expected — Chapter 6 gives the exact two commands to run at the start of every session.

Chapter 6

Network Setup (RNDIS)

Runs on: the **Raspberry Pi** for the interface commands; one step at the end runs on the **lab PC** and is marked as such.

The robot exposes an RNDIS virtual Ethernet interface over the same USB cable used for the SDK connection. The robot holds a fixed address, 192.168.42.2; the Raspberry Pi must be configured on the same subnet.

6.1 Bringing up the link

Run these two commands at the start of every session, on the Pi. The interface does not persist across Pi reboots — this is expected behavior on this hardware, not a fault, and there is no step elsewhere in this manual that avoids needing it.

```
sudo ip link set eth1 up
sudo ip addr add 192.168.42.3/24 dev eth1
```

If the interface is not named `eth1` on your system, identify it first:

```
ip -br addr
# look for eth1 / usb0 / enx<MAC> -- whichever one is meant to carry
192.168.42.x
```

Never assign 192.168.42.2 to the Pi — that address belongs to the robot.

6.1.1 If you intend to automate this over SSH

Typing the two commands by hand prompts for a password, which is fine interactively. **Any script that brings the interface up over SSH needs passwordless sudo on the Pi**, otherwise it hangs silently waiting for a prompt it cannot display — the process simply appears frozen, with no error. Check before automating anything:

```
ssh <user>@<pi> 'sudo -n true && echo "passwordless sudo OK"'
```

If it fails, grant it with `visudo` on the Pi (scope it to the two `ip` commands rather than granting blanket rights).

6.2 Verifying the link

```
ping -c 4 192.168.42.2
```

Expect 0% packet loss, sub-millisecond round-trip time once the interface is correctly up.

6.3 A quick automated alternative

Rather than typing the two commands above by hand every session, they can be scripted:

```
sudo ip link set eth1 up && sudo dhclient eth1
```

This project's own operator tooling (not covered in this manual) runs the equivalent of the first two-command form automatically as part of its first launch step, over SSH from the lab PC.

6.4 Lab PC hostname resolution

If any ROS tool on the lab PC (e.g. `rqt_image_view`) shows a black image or errors with something like `Couldn't find AF_INET address for [ubuntu]`, add the Pi's hostname to `/etc/hosts` on the lab PC:

```
echo "192.168.0.103_ubuntu" | sudo tee -a /etc/hosts
```

Unlike the RNDIS interface on the Pi, this fix is permanent on the lab PC and does not need to be repeated.

6.5 Summary of addresses used in this manual

Device	Address
Robot (over RNDIS)	192.168.42.2
Raspberry Pi (over RNDIS, to the robot)	192.168.42.3
Raspberry Pi (on the local network, to the lab PC)	192.168.0.103
Lab PC (on the local network, to the Pi)	192.168.0.100

The last two are DHCP-assigned local-network addresses and may differ on a different network; find them with `hostname -I` run on each machine (Chapter 5). Both are used again in Chapter 9 as `ROS_IP` when launching a node from that machine.

Checkpoint — do not continue until this passes. From the Pi, `ping -c4 192.168.42.2` returns 0% loss. From the lab PC, `ping -c4 192.168.0.103` reaches the Pi. If either fails, nothing in the following chapters can work, and the failure will look like a broken camera or a broken node rather than a broken link.

Chapter 7

The Camera and Robot Bridge

Runs on: the **Raspberry Pi** — this is the only machine that ever connects to the robot SDK. The verification commands at the end run from the **lab PC**.

7.1 Design: a single SDK connection

The Pi runs exactly one script that ever connects to the robot’s SDK: `rm_cam_beacon.py` (in `carolus_ws/src/robomaster_cam`). It is not just a camera capture script — it is this project’s single bridge to the entire SDK: camera stream, chassis drive, gimbal control, odometry, and the IMU subscription all go through this one script and its one connection.

This is a deliberate design choice, not an incidental one. Two independent scripts each holding their own `ep.initialize(conn_type="rndis")` connection at the same time silently blocks robot motion commands after a few calls, with no error surfaced. **Never run any other script that calls `robot.Robot().initialize(...)` while `rm_cam_beacon.py` is running.**

7.2 Starting the bridge

On the Pi, with the RNDIS link up (Chapter 6) and the Python 3.7 virtual environment active (Chapter 5). The ROS master runs on the Pi itself (Section 9.1), so `ROS_MASTER_URI` and `ROS_IP` must both point at the Pi’s own address on the lab network, `192.168.0.103` (Chapter 6) — without them `rospy` falls back to `localhost` and the node is invisible to every other machine:

```
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/Python-3.7.17/env/bin/activate
python3.7 carolus_ws/src/robomaster_cam/scripts/rm_cam_beacon.py
```

Source ROS *before* activating the venv, in the same shell: the venv only prepends its own `bin/` to `PYTHONPATH`, it does not clear the ROS environment variables sourced ahead of it.

In normal operation this is launched automatically by this project’s own operator tooling (the Tkinter launcher in `shortcuts/`, documented in `shortcuts/README.md` in the repository), over SSH from the lab PC — it is not usually started by hand.

7.3 What it connects to

```

1 self.ep = robot.Robot()
2 self.ep.initialize(conn_type="rdis")
3 self.chassis = self.ep.chassis
4 self.gimbal = self.ep.gimbal
5 self.cam = self.ep.camera
6 self.cam.start_video_stream(display=False)

```

From there it subscribes to the SDK's own push-data channels (`sub_position`, `sub_attitude`, `sub_velocity`, `sub_imu`, `sub_angle` for the gimbal), converts each callback into a ROS message, and publishes:

Topic	Content
<code>/camera/color/image_raw</code>	Camera frames (<code>sensor_msgs/Image</code>)
<code>/odom</code>	Wheel odometry, republished from the SDK
<code>/imu</code>	Raw IMU, 50 Hz requested — units: see warning
<code>/carolus/gimbal_yaw_rel</code>	Gimbal yaw relative to the chassis
<code>/carolus/gimbal_yaw_ground</code>	Gimbal yaw in the world frame (absolute)

`/imu` units are g , not m/s^2 (measured 2026-08-04). At rest on a flat floor, this topic reports `linear_acceleration.z` = -1.00664. The SDK returns accelerations in g , and `rm_cam_beacon.py` republishes them unconverted — whereas `sensor_msgs/Imu` mandates m/s^2 . Any consumer of this topic therefore sees accelerations $9.81\times$ too small under a message contract that says otherwise, and neither Kalibr nor MINS will detect it. The sign is also suspect: a REP-103 z -up frame at rest should read +9.81, not $-1.0g$. Gyroscope units and axis signs are likewise unconfirmed — a reading at rest cannot distinguish rad/s from $^\circ/s$ when the true rate is zero.

Before using this topic, verify on your own robot: gravity should read ≈ 9.81 on exactly one axis at rest, and a known slow rotation should give the expected sign and magnitude on the corresponding gyro axis. Fix scale, sign and gyro units together — correcting the scale alone yields a topic that looks right while its frame convention is still unverified.

Two further constraints: this SDK accepts only 1, 5, 10, 20 and 50 Hz for this subscription, so 50 Hz is the ceiling — below the 200–500 Hz that camera-IMU calibration tools such as Kalibr (Chapter 8) normally expect. The frequency is exposed as the `RM_IMU_FREQ` environment variable so the other legal values can be tried without editing the script.

If `/imu` is silent, check the publisher creation order before suspecting the robot.

Every ROS publisher written to from an SDK callback must be created *before* the corresponding `sub_*` subscription. The robot begins pushing data the moment the subscription is registered, and on the SDK build used here the callback runs directly in the DDS dispatcher thread: an `AttributeError` on a not-yet-created publisher kills that thread, after which *no* telemetry callback ever runs again. The symptom is indistinguishable from a robot that sends nothing — the topic is advertised, `sub_imu` returns `True`, and ROS logs nothing. This cost a full session to diagnose.

and subscribes to a small set of control topics used by this project's own higher-level tooling (not covered in this manual): `/pose` (Carolus's own output, consumed here to drive autonomous behavior), `/carolus/mode`, `/carolus/cmd_vel`, `/carolus/gimbal_vel`, `/carolus/wheels`, `/carolus/gimbal_lock`, `/carolus/gimbal_lock_period`, and `/carolus/gimbal_recenter`.

7.4 Verifying it is running

From the lab PC, with the ROS master reachable:

```
rostopic list | grep camera
rostopic hz /camera/color/image_raw
rostopic echo -n1 /odom
rostopic echo -n1 /imu
```

Measured on the Raspberry Pi with the robot connected (2026-08-04): `/camera/color/image_raw` at **30.1 Hz**, `/odom` at **15.4 Hz**, `/imu` at **47.3 Hz** against a 50 Hz subscription. Expect at least one non-empty message on `/odom` and `/imu`; if `/imu` hangs, read the second warning above before concluding anything about the robot. If the camera topic is silent, confirm the RNDIS link (Chapter 6) before suspecting the script itself — a dead network link and a dead camera stream look identical from this check alone.

Two further diagnostic lines to expect in this node’s own log, neither of which needs any setup: `[MODE]` appears once at startup, reading the robot’s current mode, forcing `FREE`, then reading it back and warning on a mismatch; `[DUTY] beacon in view NN%` appears every 30 s, the fraction of that window with a beacon actually detected — 0% with no beacon in view is correct, not a fault.

Checkpoint — do not continue until this passes. From the lab PC (not the Pi — that is the point of the check): `rostopic hz /camera/color/image_raw` reports a rate, and `rostopic echo -n1 /odom` returns one message. If the topics exist on the Pi but not from the PC, the bridge was launched without `ROS_MASTER_URI/ROS_IP` — see the launch command above.

7.5 Gimbal yaw: two signals, two different meanings

`/carolus/gimbal_yaw_rel` and `/carolus/gimbal_yaw_ground` are not interchangeable, and confusing them is a documented source of bugs in this project’s history:

- **`gimbal_yaw_ground`** is the gimbal’s heading in the world frame. The S1’s gimbal is yaw-stabilized: when the chassis rotates without an explicit gimbal command, the gimbal holds its absolute heading. This value stays essentially constant across a chassis rotation.
- **`gimbal_yaw_rel`** is the gimbal’s angle *relative to the chassis*. Because the gimbal holds its world heading while the chassis turns underneath it, this value *absorbs* the chassis’s rotation — it is not a stable “how far off is the beacon” signal across a chassis maneuver, it is a signal that actively tracks how much the chassis has turned since the gimbal last held still relative to it.

Use `gimbal_yaw_ground` when you need to know the camera’s absolute heading in the world. Use `gimbal_yaw_rel` only for what it actually is: the angle between the gimbal and the chassis at this instant, not a proxy for beacon bearing across a chassis rotation.

7.6 The internal camera vs. an external D455

This robot uses its own internal camera exclusively. If, when following any inherited reference material, you encounter instructions to launch `rs_camera.launch` or install `librealsense`, that material concerns a different camera than the one on this robot and does not apply here — see the calibration discussion in Chapter 8 for where that material remains genuinely useful (the calibration *technique*, not the camera driver).

Chapter 8

Camera Calibration

Runs on: the lab PC (or any machine with MATLAB / a Kalibr install). Images and rosbags come from the robot camera over the network.

8.1 Current values in use

The intrinsics and distortion coefficients currently loaded by `testcarolus.launch` (Chapter 9) are:

Parameter	Value
f_x	546.1957
f_y	547.0838
c_x	575.6041
c_y	372.1876
Distortion (plumb_bob: k_1, k_2, p_1, p_2, k_3)	-0.1479, 0.1452, -0.0023, 0.0025, 0.0070

These come from `robomaster_cam/config/camera_info.yaml` (`camera_name: robomaster_s1`, 1280×720), validated pragmatically — Carolus produces a stable, coherent pose with these values in practice. They are **not** the result of a formal metric calibration campaign (checkerboard/AprilGrid target, dedicated calibration tool run to convergence with a reported reprojection error). If you are setting up a new unit, or want a rigorous calibration, follow the procedure below rather than reusing these values — they are specific to this project’s camera and are not guaranteed to transfer to a different physical unit.

8.2 Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox

This procedure has not yet been run against this project’s own camera — the values in the previous section are what is actually loaded today. It is included here as the recommended path for whoever runs the next calibration, based on a direct comparison against Kalibr on a comparable camera (Section 8.3).

1. Print a checkerboard target with known square size (25 mm squares is a proven working size for this class of camera).
2. Capture a set of images of the board with the robot’s camera (Chapter 7): at least 10 images, 15 or more for better accuracy. The board should occupy roughly 20–25% of the frame area, and be

tilted more than 45° from the camera’s optical axis in each shot to avoid detection errors. Capture from a wide variety of positions and orientations across the field of view.

3. In MATLAB (desktop or MATLAB Online, no additional software required): open the **Camera Calibration app** (Apps tab), add the captured images, select the checkerboard pattern with the correct square size, and enable **tangential distortion** and **three radial distortion coefficients** (this camera class shows moderate wide-angle distortion that the default two-coefficient model under-fits).
4. Run the calibration. Export the resulting coefficients to the MATLAB workspace and generate the calibration script — this script contains the intrinsic matrix, distortion coefficients, and reprojection errors in a form that can be transcribed directly into `testcarolus.launch`’s `fx/fy/cx/cy/distortion` parameters (Chapter 9).

8.3 Alternative / complementary method: Kalibr

Kalibr (ETH Zürich) is the standard tool if a camera–IMU joint calibration is needed (required for any future MINS/visual-inertial integration, not needed for Carolus’s own pose estimation, which does not use the IMU).

Not executed here. The camera–IMU half of this procedure has never been run on this robot. The camera-only calibration below is unaffected and is the part that matters for Carolus. Two constraints to know before planning a camera–IMU session on this platform: the SDK caps the IMU subscription at 50 Hz, well below the 200–500 Hz Kalibr expects for a well-conditioned solution; and `/imu` publishes accelerations in g rather than the m/s^2 the message type mandates (Chapter 7), with gyroscope units and axis signs still unconfirmed. Correct the units and verify the frame first — Kalibr will not detect either error, and every noise parameter it produces would be wrong by the same silent factor.

8.3.1 Target

A 6×6 AprilGrid, tag size 0.089 m, tag spacing $0.3 \times$ tag size (an $\sim 0.8 \times 0.8$ m printed board). Measure the actual printed square size and use the measured value, not the nominal one — even a $\sim 1\%$ sizing error biases the recovered scale.

```
1 target_type: 'aprilgrid'
2 tagCols: 6
3 tagRows: 6
4 tagSize: 0.089          # metres -- MEASURE your own printed board
5 tagSpacing: 0.3         # ratio of gap to tag size
6 codeOffset: 0
```

8.3.2 Installing Kalibr

```
cd ~/camera_ws/src
git clone https://github.com/ethz-asl/kalibr.git
cd ~/camera_ws
rosdep install --from-paths src --ignore-src -y
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-
generator \
    build-essential python3-catkin-tools
sudo apt-get install libv4l-dev doxygen python3-igraph
```

```

sudo apt install python3-pip
pip install wxPython      # can take a long time -- be patient
catkin_make -DCMAKE_BUILD_TYPE=Release
source ~/camera_ws/devel/setup.bash

```

Do not separately clone `numpy_eigen` or `Schweizer-Messer` — Kalibr bundles its own copies, and cloning them independently into `src/` produces duplicate-package build errors. If you hit `ModuleNotFoundError: No module named 'sm'` after building, verify with:

```
python3 -c "import sm; print(sm.__file__)"
```

and if it fails, remove any independently-cloned copy of `Schweizer-Messer`, then rebuild:

```

rm -rf build/ devel/ install/
catkin_make -DCMAKE_BUILD_TYPE=Release

```

8.3.3 Capturing data and running the calibration

Record a rosbag while moving the camera (or the robot itself, for a joint camera–IMU calibration) through the AprilGrid’s field of view for 60–90 seconds, exciting all six degrees of freedom (translate near/far/side to side, rotate about all three axes, not just yaw).

```
rosbag record -O calib_data.bag /camera/color/image_raw
```

Run the calibration:

```

export KALIBR_MANUAL_FOCAL_LENGTH_INIT=1
kalibr_calibrate_cameras \
  --bag calib_data.bag --target target.yaml \
  --models pinhole-radtan \
  --topics /camera/color/image_raw \
  --approx-sync 0.01

```

If Kalibr reports `Initialization of focal length failed`, provide a manual seed value (600 is a reasonable starting guess for a 640×480 -class image; scale proportionally for other resolutions).

8.3.4 Reading the result

Kalibr writes three files: a `.txt` summary, a `.pdf` of reprojection-error plots, and a `.yaml` with the calibration parameters. **Target: reprojection error under 0.2–0.5 pixels.** In practice, a completed run can land a little outside that target (observed: 0.59–0.67 px std on a comparable camera) without necessarily being unusable — treat the target as a goal to aim for, not a hard pass/fail gate, and judge the calibration by whether downstream pose estimates are stable and physically plausible.

8.4 Camera–IMU time offset

If a Kalibr camera–IMU joint calibration is run, its output `timeshift_cam_imu` is in **seconds**, with the convention $t_{\text{imu}} = t_{\text{cam}} + \text{shift}$. This matches OpenVINS’s own internal convention exactly — the value can be used directly with no sign flip if it is later consumed by an OpenVINS-based filter.

Chapter 9

Building and Launching Carolus

Runs on: the Raspberry Pi 4B for actually running Carolus — this is where the node executes, full stop, not one of two equally valid options. The **lab PC** build is documented too, but only as a development environment for editing and compiling the C++ before deploying it to the Pi; Section 9.1 gives the measurements behind why operation and development are split this way.

Carolus (the ROS package name is `carolus_astrobee`) has no SDK connection of its own — it consumes `/camera/color/image_raw` (published by `rm_cam_beacon.py`, Chapter 7) and publishes a pose on `/pose`, so it only needs to reach the ROS master, not the robot. That is *why the lab PC can build and run this code at all* despite never touching the robot directly — not a reason to operate it there, which the next section settles.

9.1 Where Carolus runs

The Pi runs Carolus, always — autonomy, not speed, is the decisive argument. A robot whose pose estimator lives on another machine stops working the moment the link does; that is a property of the architecture, not a number a future optimisation could fix. The 2–3× camera input rate and the absence of build workarounds (below) are real, but secondary. **The lab PC builds and edits the same code but never operates the robot with it** — a development convenience, kept because compiling and iterating C++ is faster on a workstation than over SSH and the operator GUI already runs there.

9.1.1 The two machines, side by side

	Raspberry Pi 4B (where Carolus runs)	Lab PC (development only)
Robot autonomy	Complete. No network link is needed to estimate pose.	None. Every pose depends on a live link; the robot cannot leave the PC's network reach.
Camera input rate (measured)	30.1 Hz — frames never leave the machine that captured them.	10–15 Hz — every frame is an <i>uncompressed</i> <code>sensor_msgs/Image</code> at 1280×720 (≈ 2.76 MB) crossing the network first.
Build complexity	Ubuntu 20.04 = ROS Noetic's officially supported target. Standard <code>catkin_make</code> , no workarounds (Section 9.3).	On Ubuntu 22.04: four build-time fixes plus one runtime fix (Section 9.4). On 20.04, identical to the Pi.
Development comfort	Editing and rebuilding C++ over SSH is slower.	Better. A workstation compiles faster and edits more comfortably; GUI tooling runs here.
Visual-inertial work (MIN-S/OpenVINS)	Inside the upstream tested set (Ubuntu 18/20, Melodic/-Noetic).	On 22.04, outside it.

9.1.2 Measured, 2026-08-04 — and what the numbers do *not* say

Same robot, same beacon at 1.00 m, same afternoon — Carolus run once on each machine and `/pose` measured both times:

	Carolus on the lab PC	Carolus on the Pi
<code>/pose</code> , beacon in view	2.19 Hz	13.04 Hz
<code>/postprocessed/image</code>	—	14.35 Hz
<code>/camera</code> at the Pi's publisher	30.05 Hz	16.52 Hz
Pi CPU	52–61% idle	12.3% idle , load 5.37/4

A factor of **5.95**, from an uncompressed 1280×720 `sensor_msgs/Image` (≈ 2.76 MB) crossing the network on the PC versus never leaving the machine on the Pi. The other rows say where the limit moved to: `/pose` 13.0 Hz, `/postprocessed` 14.4 Hz and the camera at 16.5 Hz are nearly the same number, and the camera itself drops from 30.05 Hz to 16.52 Hz once Carolus runs beside it, with the Pi at 12.3% idle — **the bottleneck on the Pi is now CPU**, camera bridge and Carolus competing for it. If more pose rate is needed, look there: compressed frames, or a lower capture resolution.

Verify which machine you are measuring. Both numbers above are /pose rates with the same beacon, and the only difference is where the node ran. It is easy to get this wrong: the ROS master lives on the Pi in both cases, so a Carolus running on the workstation still looks Pi-side from the topic list. Confirm with `pgrep -f carolus_astrobee` on the Pi before attributing a rate to it.

Two older figures circulate and neither is a pose rate: a ~ 24 frames/s figure for the Pi was counted **with no beacon in view**, so no P4P solve ran on any frame, and a ~ 2.5 frames/s figure for the lab PC used a different counting method again. Use the table above, not those.

The Pi already had Ceres 1.14, Eigen 3.3.7 and OpenCV 4.2 installed; `catkin_make` compiled and linked directly on `aarch64`. Available memory (7.6 GB) was never a constraint.

9.1.3 A note on visual-inertial navigation

MINS has been run on this Raspberry Pi, both on its own simulation and on data recorded from this robot. It builds on `aarch64` and produces a *consistent* estimate on the simulation (RMSE 0.113° / 0.082m , NEES 0.9 and 1.4), with no divergence, at **$0.3\text{--}0.4\times$ real time** — under a sensor load far heavier than this robot’s (IMU at 200 Hz, *two* cameras, and a LIDAR).

On a 60 s recording of *this* robot’s `/imu` and `/camera/color/image_raw`, it processed the whole bag at **$0.7\times$ real time**: CAM 118 ms, IMU 3 ms, 122 ms of compute per 88 ms of data. So “**MINS does not run on a Raspberry Pi**” is **not correct** — it runs, on this robot’s own data. What is correct is that **it does not reach real time here**, even with one camera, no LIDAR and a 50 Hz IMU, and that essentially all of the cost is the camera update. The $0.7\times$ is also optimistic: `rosviz` record throttled the sources to 16 Hz of camera against 30 Hz live.

Two practical points if you repeat this. MINS’s live-subscribe entry point is **ROS 2 only** — in `run_subscribe.cpp` the `ROS_AVAILABLE == 1` branch contains two `#include` lines and nothing else, so under ROS Noetic the binary initialises, subscribes to nothing, and exits immediately even with data flowing. Use the `bag` binary and a recorded rosbag, which is the path `subscribe.launch`’s own hard-wired `rosviz play` node implies. And do not quote pose accuracy from such a run unless the camera–IMU extrinsic has been calibrated and the IMU units corrected (Chapter 7) — with a placeholder extrinsic and accelerations in *g*, the filter diverges grossly while the timing figures stay valid.

9.2 Getting this project’s code onto the machine that will build it

Clone the project’s repository onto whichever machine will build Carolus — **normally the Raspberry Pi** (Section 9.1), over SSH from the lab PC (Chapter 5). Cloning into a directory named `carolus_ws` gives a standard catkin workspace layout, with the ROS packages under `src/`: `src/carolus_node/` (the launch files and configuration), `src/libuvgs_astrobee/` (Carolus’s own C++ source, already patched for this robot), `src/ff_msgs/` (a message-definition dependency), `src/robomaster_cam/` (the Pi-side bridge, Chapter 7), plus `cmake_shims/` (Ubuntu 22.04 build fixes, see below) and `shortcuts/` (this project’s operator tooling, not covered in this manual) at the top level:

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git carolus_ws
# or, over HTTPS -- the repository is public, no credentials needed:
# git clone https://github.com/Arracheur2Bonnet/robotmaster.git carolus_ws
```

Repository web address: <https://github.com/Arracheur2Bonnet/robotmaster>

The repository is public. Both clone commands above work as-is, from any machine, with no

access request and no credentials needed to read it. The SSH setup below is still worth doing even so — it avoids retyping credentials on every push, and it is required the moment you are given write access.

9.2.1 Setting up an SSH key

Cloning over SSH (the first command above) needs an SSH key registered against *your own* GitHub account. This is a one-time setup per machine, and it is what replaces the practice — found in some older inherited documents — of pasting a shared personal access token directly into a clone command. A shared token grants whoever holds it the full access rights of the account that created it, cannot be attributed to an individual, and stays valid in every copy of the document that carries it.

1. Generate a key on the machine that will clone the repository (the Raspberry Pi and the lab PC each need their own):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept the default path (~/.ssh/id_ed25519).
# A passphrase is optional but recommended.
```

2. Display the **public** half of the key — this is the only part that ever leaves the machine. Never share the file without the `.pub` extension:

```
cat ~/.ssh/id_ed25519.pub
```

3. On GitHub: *profile photo* → *Settings* → *SSH and GPG keys* → *New SSH key*. Give it a title identifying the machine (e.g. “RoboMaster Pi”), paste the full line printed above, and save.
4. Verify the key works before attempting the clone:

```
ssh -T git@github.com
```

A successful setup answers with `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.` — that message is the expected result, not an error.

If you prefer HTTPS to SSH, use the second clone command instead and authenticate with your GitHub username and a personal access token *that you generate for your own account* — created under *Settings* → *Developer settings* → *Personal access tokens*, scoped to repository read access, and never shared with anyone else or written into a document.

9.3 Building on Ubuntu 20.04 (the Raspberry Pi — normal case)

This is the normal path: the Raspberry Pi runs Ubuntu 20.04, ROS Noetic’s officially supported target, so the build is standard and needs no workarounds. (A lab PC also on 20.04 builds identically.)

```
cd ~/carolus_ws
catkin_make --pkg ff_msgs # build this dependency first, avoids ordering
issues
catkin_make
source devel/setup.bash
```

That is the whole build. Skip the 22.04 section entirely — it exists only for a lab PC on an unsupported OS — and go to Section 9.5.

On a Pi, expect the compile to take several minutes. If memory is tight on a 4 GB model, pass `-j2` to limit parallel jobs.

9.4 Building on Ubuntu 22.04 (lab-PC fallback only)

Skip this section unless you are building on a lab PC running Ubuntu 22.04. None of it applies to the Raspberry Pi.

ROS Noetic officially targets Ubuntu 20.04 only. On 22.04, four system-level fixes are needed before `catkin_make` succeeds, and a fifth at launch time. They are recorded because this project's lab PC was upgraded to 22.04 unintentionally and the workarounds took real effort to find — not because this is a recommended configuration.

9.4.1 Fix 1: glibc symlinks

`librt`, `libpthread`, and `libdl` were merged into `libc.so.6` starting with glibc 2.34 (shipped in 22.04); older binaries and build systems still look for the separate files.

```
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/librt.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/libpthread.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/libdl.so
```

9.4.2 Fix 2: libpython3.8 symlink

Referenced by `rospack` (an official ROS Noetic package):

```
sudo ln -sf /usr/lib/x86_64-linux-gnu/libpython3.8.so.1 /usr/lib/x86_64-linux-gnu/libpython3.8.so
```

9.4.3 Fix 3: Boost 1.71 (real libraries, not symlinks)

ROS Noetic's official binaries need Boost 1.71; 22.04 ships 1.74, which is ABI-incompatible for the libraries ROS actually links (`regex`, `filesystem`). Symlinking will not work here — the real 1.71 `.so` files are needed:

```
# Download from http://old-releases.ubuntu.com/ubuntu/pool/{main,universe}/b/boost1.71/
# the following packages: libboost-{chrono,date-time,filesystem,program-options,
# regex,system,thread}1.71.0

dpkg-deb -x <package>.deb <extract_dir>
# copy each extracted .so.1.71.0 file into /usr/lib/x86_64-linux-gnu/
sudo ldconfig
```

Pitfall: if a symlink already exists at the destination path (pointing at the 1.74 library), a plain `cp` writes *through* the symlink and corrupts the real system 1.74 file. Always remove the symlink first:

```
sudo rm -f /usr/lib/x86_64-linux-gnu/libboost_<name>.so.1.74.0 # if it is a symlink
# then copy the real 1.71 file to that path
```

If this happens anyway, recovery is a straightforward package reinstall:

```
sudo apt install --reinstall libboost-{chrono,date-time,filesystem,program-  
options,regex,system,thread}1.74.0
```

9.4.4 Fix 4: ICU 67

A dependency of Boost 1.71's regex component; 22.04 ships 66/70, not 67.

```
# Download libicu67 from old-releases.ubuntu.com  
# copy libicuuc.so.67*, libicui18n.so.67*, libicudata.so.67* into /usr/lib/  
x86_64-linux-gnu/  
sudo ldconfig
```

9.4.5 Building, after the four fixes

```
cd ~/carolus_ws  
rm -rf build/ devel/  
catkin_make --pkg ff_msgs  
catkin_make  
source devel/setup.bash
```

9.4.6 Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)

A binary compiled on 22.04 links against `liblog4cxx.so.12` and `libtinyxml2.so.9`; ROS Noetic's own libraries (`libroscconsole.so`, `libimage_transport.so`) expect `.so.10/.so.6`. Both versions load at runtime and the 22.04 version wins global symbol resolution, which segfaults inside `ros::console::initialize()`. The fix, scoped to the affected node in its launch file (already applied in the launch file below, and in `ekf_lite.launch`):

```
1 <env name="LD_PRELOAD"  
2   value="/usr/lib/x86_64-linux-gnu/liblog4cxx.so.10:/usr/lib/x86_64-linux-  
   gnu/libtinyxml2.so.6" />
```

Apply the same pattern to any other C++ node recompiled under 22.04 that segfaults the same way on startup.

9.4.7 If an additional ROS package needs to be built from source

Packages unavailable via `apt` on 22.04 (e.g. `robot_localization`, for planned EKF sensor-fusion work) can fail with CMake errors about a missing `<pkg>Config.cmake` or a nonexistent `orocos_kdl` include path — seven packages (`eigen_conversions`, `tf2_geometry_msgs`, `kdl_parser`, `kdl_conversions`, `robot_state_publisher`, `tf2_kdl`, `tf_conversions`) are broken this way by the 20.04→22.04 packaging difference. Without root access to fix the real files under `/opt/ros/noetic/`, the working solution is a set of locally corrected copies in `carolus_ws/cmake_shims/<pkg>_fix/`, given priority via `list(PREPEND CMAKE_PREFIX_PATH ...)` in the `CMakeLists.txt` of whichever package is being built.

Stale-cache pitfall: if a shim is created only after a first failed build attempt, `<pkg>_DIR` may stay cached with the old broken path. Clear just that variable rather than wiping the whole build:

```
cmake -U <pkg>_DIR
```

9.5 Launching Carolus

Carolus's parameters are split in two, so the node can run on other robots without editing any code:

- `src/carolus_node/config/robomaster_s1.yaml` — everything specific to *this* robot and *this* beacon: camera intrinsics, distortion, beacon geometry, blob-detection tuning.
- `src/carolus_node/launch/carolus.launch` — the generic launch file. It contains no hardware values at all; it loads a profile and wires up topics.

`testcarolus.launch` is the RoboMaster S1 entry point: it selects the S1 profile, adds an S1-specific static transform, and delegates to `carolus.launch`.

9.5.1 The hardware profile

The values below are this robot's. **Every one of them must be re-measured for different hardware** — see Chapter 8 for the camera intrinsics, and the note on `known_points` at the end of this chapter for the beacon.

```
1 # --- Camera intrinsics (MEASURE THESE FOR YOUR OWN CAMERA) ---
2 fx: 546.1957
3 fy: 547.0838
4 cx: 575.6041
5 cy: 372.1876
6 distortion: "-0.1479,0.1452,-0.0023,0.0025,0.0070" # k1 k2 p1 p2 k3
7
8 # --- Beacon geometry (MEASURE THIS FOR YOUR OWN BEACON) ---
9 # Four LED positions in metres, Carolus's left-handed frame. One point is
10 # deliberately off the plane of the other three -- P4P needs that.
11 known_points: "0.0825 0.0 0.0 -0.0825 0.0 0.0 0.0 0.072 0.0 0.0 0.0 0.0555"
12
13 # --- Blob detection ---
14 min_circularity: 0.6
15 saturation_threshold: 80
16 min_area: 8.0
17 max_area: 1800.0
18 max_distance_lim: 1000.0
19 lb_hue: 90.0 # OpenCV 0-180 scale, NOT 0-360. Blue LEDs here.
20 ub_hue: 140.0
21
22 # --- Preprocessing ---
23 kernel_size_gaussian: 3
24 kernel_size_morph: 3
25 image_threshold: 190
26
27 # --- Outlier rejection (moving window over successive poses) ---
28 filter_size: 7
29 translation_threshold: 0.5
30 rotation_threshold: 0.3
31 fifo_on: true
32
33 # --- Execution ---
34 num_threads: 1
35 queue_size: 10
36 benchtest: true
```

```

37
38 # --- Camera selection (Astrobee-era flags, kept for compatibility) ---
39 nav_cam: true
40 dock_cam: false
41 fisheye: false
42 fov: true
43 mono: false
44 bot_name: "wannabee" # inert legacy label, no functional effect

```

9.5.2 Launching

On the Raspberry Pi (the normal case). The `ubuntu2204_preload:=false` argument is required: the LD_PRELOAD workaround it disables exists only for the 22.04 library mismatch (Fix 5), and it hardcodes x86_64 paths, so leaving it on would be not merely unnecessary but wrong on aarch64.

```

# On the Pi, with the camera bridge already running (Chapter 6), in a
# SEPARATE shell -- ROS_MASTER_URI/ROS_IP are per-shell, not inherited:
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/carolus_ws/devel/setup.bash
roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false

```

Carolus and the camera bridge then both run on the Pi, and no image crosses the network — which is what makes the robot usable away from the lab PC, and what gives the camera input rate measured in Section 9.1.2.

On a lab PC running Ubuntu 22.04 (development only, per Section 9.1), the master is still on the Pi — `ROS_MASTER_URI` points there, `ROS_IP` is the PC's own address (Chapter 6), and the preload argument is omitted so it stays enabled:

```

export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.100
roslaunch carolus_node testcarolus.launch

```

On a different robot entirely, copy the profile, substitute your own measured values, and point Carolus at your own camera topic — and set `ROS_MASTER_URI/ROS_IP` for *that* robot's own network, not this project's addresses above. No source file and no launch file needs editing:

```

roslaunch carolus_node carolus.launch \
  config:=$(rospack find carolus_node)/config/my_robot.yaml \
  camera_topic:=/my/camera/image_raw \
  ubuntu2204_preload:=false

```

The `robomaster_cam` package (Chapter 7) is the DJI SDK bridge for this specific robot and is **not** needed on other hardware — supply the camera stream however your own platform already does.

9.5.3 The pose timestamp: `stamp_from_acquisition`

`/pose`'s header is stamped with the ACQUISITION time of the image it was computed from, not the time it was published — **leave this at its default of true**. The alternative (`stamp_from_acquisition:=false`) reproduces the old behaviour, where every pose is stamped `ros::Time::now()` at the moment Carolus finishes computing it. That difference is not cosmetic: a fusion filter reads `header.stamp` to place the measurement in time, so the old behaviour hands it a systematic offset it has no way to detect, and

it also makes the pose's real latency unmeasurable from ROS alone — the gap between capture and publication silently disappears into "now".

```
roslaunch carolus_node testcarolus.launch stamp_from_acquisition:=false
```

The only legitimate reason to set it **false** is if the image timestamps come from a machine whose clock is not synchronised with whatever consumes `/pose` — and even then, fix the clock sync rather than the stamp. **Check this before trusting it:** the Pi's system clock is not guaranteed to be NTP-synchronised (observed drifted by several days on this project, 2026-08-10), and a drifted clock corrupts this exact mechanism silently, the same way the old publish-time stamping did.

9.6 Verifying it is running

```
roslaunch carolus_node testcarolus.launch stamp_from_acquisition:=false
rostopic info /carolus_astrobeerex
rostopic hz /pose
rostopic hz /postprocessed/image
```

With a beacon visible to the camera, `/pose` publishes a `geometry_msgs/PoseStamped`. Visualize blob detection in RViz by adding an **Image** display on `/postprocessed/image`: correctly detected blobs appear as small circles over the live feed.

Check the latency, not just the rate. A high `/pose` rate says nothing about whether `stamp_from_acquisition` landed correctly (previous section). In the camera bridge's own log (`rm_cam_beacon.py`, Chapter 7), every 50 poses prints `[LATENCY] /pose NN ms (mean over 50 poses)` — a plausible value confirms acquisition-time stamping and synchronised clocks. A value outside `-1-5000 ms` instead logs `[LATENCY] implausible (NN ms)` — `/pose` is probably stamped at publication time, not acquisition, naming the same two causes: an old binary running without the flag, or the two machines' clocks disagreeing.

Checkpoint — the core pipeline is now complete. With a powered beacon in the camera's view, `rostopic hz /pose` reports roughly 13Hz on the Pi. No beacon in view means no pose at all — that is correct behaviour, not a fault. Once this passes, go to Chapter 10 to actually drive the robot.

9.7 Beacon geometry and blob-filtering parameters

The parameters most likely to need tuning for a new beacon or lighting condition:

Parameter	Meaning
<code>min_circularity</code>	Rejects non-round contours; closer to 1 is stricter.
<code>saturation_threshold</code>	Rejects low-saturation pixels (in HSV) before hue filtering.
<code>min_area, max_area</code>	Rejects contours outside this pixel-area range.
<code>max_distance_lim</code>	Rejects candidate 4-blob groups whose pairwise distances exceed this.
<code>lb_hue, ub_hue</code>	Hue range the beacon's LEDs must fall within, in OpenCV's 0-180 scale , not 0-360.

The hue scale is worth stating explicitly, because getting it wrong silently kills all detection. OpenCV stores hue on 0-180 (half the usual 0-360 degree circle, so it fits in one byte), and the node compares the blob's mean hue against `lb_hue/ub_hue` on that same 0-180 scale. Rough

reference points: red $\approx$ 0–10 and 170–180, green $\approx$ 50–80, blue $\approx$ 100–140. This project’s beacon uses blue LEDs, hence the 90–140 window in the launch file. If you enter a value from the 0–360 convention (e.g. 240 for blue), every blob is rejected and the node runs normally while never publishing a pose.

`known_points` is the beacon’s four LED positions in metres, in Carolus’s own left-handed frame, with one point deliberately off the plane of the other three (needed for P4P to resolve pose unambiguously).

These are a physical measurement of this project’s specific beacon, not a universal constant — the same caveat as the camera intrinsics in Chapter 8. A beacon built to different dimensions needs these four points re-measured and substituted; reusing this project’s values against a differently-sized beacon will make the P4P solve fail silently or produce a plausible-looking but wrong pose.

Chapter 10

Operating the Robot: the Launcher

Runs on: the **lab PC** — the launcher is a Tkinter GUI that starts everything else over SSH. The robot and the Pi must both be on.

Everything up to here produced a pose on `/pose`. This chapter is how the robot is actually driven. The tool is `shortcuts/carolus_launcher.py`, in the repository you cloned; it starts the whole stack with no commands to type, shows a live dashboard, and pilots the robot from the keyboard.

```
python3 shortcuts/carolus_launcher.py
```

Fuller reference: `shortcuts/README.md` in the repository documents every button, panel and keyboard binding, and is kept current with the code. This chapter covers what a first-time operator needs.

10.1 The launch tabs, in order

Each tab is one process. They are started in order — the first two wait for a concrete signal before unlocking the next, rather than a fixed delay.

Tab	What it starts	Unlocks when
T1 roscore [Pi]	SSH: <code>eth1 up + roscore</code> on the Pi	port 11311 is open (60 s timeout)
T2 Camera [Pi]	SSH: <code>rm_cam_beacon.py + cam_view_helper.py</code>	<code>/camera/color/image_raw</code> is publishing (60 s timeout)
T3 Carolus [Pi]	SSH: <code>roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false</code>	manual — no wait
T4 TF [Pi]	SSH: <code>carolus_tf_broadcaster.py</code>	manual. Republishes <code>/pose</code> as a TF (<code>camera_link</code> → <code>beacon_observed</code>). Not needed by the SEARCH/ALIGN/APPROACH pipeline, which reads <code>/pose</code> directly
T5 Docking [PC]	local: <code>beacon_docking.py</code>	manual (see Section 10.6)
T6 MINS [Pi]	SSH: MINS's own <code>simulation.launch</code> , in a separate sandbox workspace	independent — runs its own master, unrelated to the pipeline above

T1–T3 are the working stack. T4 is for validating orientation and for future sensor-fusion work; T5 is the docking behaviour; T6 is unrelated experimental work. A normal session is T1, T2, T3.

Every tab label names its machine (T1 roscore [Pi], T5 Docking [PC]). This is deliberate: the ROS master lives on the Pi in every configuration, so nothing else on screen distinguishes a node running on the Pi from one running on the workstation — and a rate measurement was once attributed to the wrong machine for exactly that reason.

10.2 Driving the robot by hand

Click **MODE: AUTO** to switch to **MODE: MANUAL** (the button turns orange). Two key blocks appear and light up as keys are pressed.

Chassis — ZQSD	Gimbal — numpad
Z forward S reverse	8 pitch up 2 pitch down
Q rotate left D rotate right	4 yaw left 6 yaw right 5 stop
0.20 m/s linear, 20°/s angular. Stops automatically when keys are released.	30°/s pitch, 40°/s yaw. Works with Num-Lock on or off.

Clicking **MODE** again returns to **AUTO** and stops both immediately.

If the keys stop responding, move the mouse over the launcher window. T1 and T3 can open external terminal windows that steal the system's keyboard focus. Hovering over the launcher reclaims it automatically. This is not a bug in the robot — the commands are simply not reaching the application.

The numpad does nothing while LOCK is on. LOCK takes exclusive control of the gimbal; the chassis keys keep working normally. Turn LOCK off to get manual gimbal control back.

10.3 The main controls

Control	What it does
MODE: AUTO/MANUAL	Switches between the autonomous state machine and keyboard piloting.
LOCK	Periodically re-centres the gimbal on the beacon. Period configurable in seconds. Ignores corrections below a 5° deadband and skips any beyond 45° (an implausible pose).
RECENTER CAM	Returns the gimbal to its base position.
CAM PREVIEW	Toggles the camera view. Off by default — it costs network bandwidth and GUI smoothness, so turn it on only when you need to see the image.
LOCATE	Localises the beacon without advancing. Activates automatically 500 ms after T2 confirms the camera is ready.
Kill	Stops everything and <i>verifies</i> the remote processes actually died.
F11 / Escape	Fullscreen on / off.

The **dashboard** shows the current state (SEARCH / APPROACH / STOP / LOCATE / MANUAL), the beacon distance, the robot's battery, and the Pi's own temperature, load and RAM (refreshed every 20s). The Pi's temperature turns amber at 65 °C and red at 75 °C, ahead of the Pi 4B's 80 °C thermal throttle.

The **beacon indicator** reads BEACON: DETECTED (green) or BEACON: LOST (red), and a small minimap shows where the beacon sits *within the camera frame* — green when centred within $\pm 3^\circ$, orange otherwise.

10.4 Session logs on disk

Every line shown in any tab is also written to `shortcuts/logs/session-YYYY-MM-DD-HH-MM-SS.log`, one file per launcher start, each line tagged with its tab. The on-screen tabs are truncated to 300 lines and lost on exit, so the file is the only durable record — and it makes questions answerable after the fact:

```
grep '\[T2\].*\[LOCK\]' shortcuts/logs/session-*.log
grep '\[T3\].*\[LATENCY\]' shortcuts/logs/session-*.log
```

10.5 Shutting down, and why it matters

Use the launcher's Kill, or close the window — both clean up the remote processes and re-read the Pi to confirm each one actually died, logging TOUJOURS VIVANT for any survivor rather than failing silently.

An orphaned `rm_cam_beacon.py` breaks the next session. It keeps the robot's single SDK connection open, so the next launch hits the double-connection failure — motion commands stop responding with no error anywhere. If you ever kill processes by hand, **bracket the pattern:** `pkill -f "[r]m_cam_beacon"`. Without the bracket, the remote shell's own command line contains the pattern and `pkill` can kill that shell before it reaches the real target, non-deterministically and with nothing reported.

10.6 Beacon docking (T5)

Status: works, with a known limitation — read this before running it near anything fragile.
The docking behaviour drives the robot to a fixed distance from the beacon (0.20 m) in three phases: align the gimbal to the beacon, rotate the chassis until it agrees with the gimbal, then drive forward.

The **DOCKING BALISE** panel has four buttons:

Button	What it does
CALIBRATE	Step 1 of a two-step calibration: measures the beacon face-on.
CAL STEP 2	Step 2: measures again after <i>you</i> have pivoted the robot roughly 30–45°. No time pressure between the two steps.
START	Runs the docking sequence.
ABORT	Stops it.

The buttons only unlock once the node has reported its own status at least once — commands sent before that used to be silently dropped.

Known limitation, unresolved: nothing corrects the chassis heading *during* the drive — only the camera tracks the beacon. A run can therefore finish close to the beacon but off-centre. A collision has happened once on this project, caused by a silent gimbal-probe failure that let the chassis align to the wrong reference (since fixed, with a retry and an abort guard).
Run docking with clear space around the robot, and keep a hand near ABORT.

Chapter 11

Troubleshooting

Runs on: wherever the symptom appears. Each entry names the machine to check on.

Work down from the symptom. Each step either fixes the problem or rules out a layer — the ordering matters, because a dead network link and a dead node look identical from most checks.

11.1 Nothing responds: the robot ignores every command

1. **Did the robot chime twice at power-on?** One chime means the root hack is not active and the SDK will not answer at all (Chapter 3). Power-cycle and listen again.
2. **Is a second script holding the SDK?** Only `rm_cam_beacon.py` may connect. On the Pi: `pgrep -af "[r]m_cam_beacon"` — more than one result, or an orphan from a previous session, causes exactly this symptom with no error message anywhere. Kill it and relaunch.
3. **Is the RNDIS link up?** From the Pi: `ping -c4 192.168.42.2`. It does not survive a Pi reboot (Chapter 6).

11.2 The Pi will not turn on

The cable is almost certainly in the Pi's own POWER IN port instead of the UPS HAT's port underneath. Check that the HAT's blue charge LED is lit (Section 5.1).

11.3 `rostopic` from the lab PC finds nothing

1. **Is the node actually running?** On the Pi: `pgrep -af "[r]m_cam_beacon"`.
2. **If it is running but invisible from the PC**, the shell that launched it was missing `ROS_MASTER_URI/ROS_IP` — `rospy` falls back to `localhost` and the node is reachable only from the Pi itself. Relaunch with both exported (Chapter 7). This is the single most common cause.
3. **Is your own shell pointed at the right master?** On the PC: `echo $ROS_MASTER_URI` should read `http://192.168.0.103:11311`.
4. **A black image or an `AF_INET` error** instead of no topic at all is hostname resolution — add the Pi to `/etc/hosts` on the PC (Chapter 6).

11.4 The camera publishes but `/pose` never does

1. **Is a beacon actually in view, powered on?** Check `[BEACON] status=` in the T2 log, or the launcher's beacon indicator. `[DUTY] beacon in view 0.0%` confirms it is not.
2. **Is Carolus running?** On the Pi: `pgrep -f carolus_astrobees`.
3. **Are the hue bounds right for your LEDs?** This is the classic silent failure: the node runs, looks healthy, and rejects every blob. `lb_hue/ub_hue` are on **OpenCV's 0–180 scale**, not 0–360 — entering 240 for blue rejects everything. See the end of Chapter 9.
4. **Are all four LEDs visible?** P4P needs four points; three is not enough and produces nothing rather than a degraded pose.

11.5 The keyboard does not drive the robot

1. **Is the launcher in MANUAL mode?** The button must read `MODE: MANUAL` in orange (Chapter 10).
2. **Move the mouse over the launcher window** — an external terminal opened by T1/T3 has probably stolen keyboard focus.
3. **Numpad specifically dead?** LOCK is on and holds the gimbal exclusively. The chassis keys should still work.

11.6 `/imu` is silent

Do not suspect the robot first. If a ROS publisher is created *after* its SDK subscription, the first callback raises inside the SDK's dispatcher thread and kills it — after which no telemetry callback ever fires again, with the topic still advertised and nothing logged. See the second warning box in Chapter 7.

11.7 The build fails

On the Pi (Ubuntu 20.04) the build is standard; a failure there is usually memory — pass `-j2`. On a lab PC running Ubuntu 22.04, the five fixes in Section 9.4 are all required, and a segfault at launch rather than at build time is Fix 5 (the `LD_PRELOAD`).

11.8 Messages you can safely ignore

Three appear routinely on a healthy system and have each cost someone an afternoon:

- `bash: .../catkin_ws/devel/setup.bash: No such file or directory` — a leftover line in a shell profile. Harmless.
- `WARNING: disk usage in log directory [...] is over 1GB` — accumulated ROS logs. `rosclean purge` when convenient.
- `imageio 2.35.1 has requirement pillow>=8.3.2, but you'll have pillow 7.0.0` — from `myqr`, a transitive dependency of the SDK used only for Wi-Fi QR codes, which this RNDIS setup never uses.